

Batch properly if you self-host:

vLLM's continuous batching and PagedAttention give several times the throughput on the same card. Quantised builds, as I wrote in July, usually deliver most of the quality at a fraction of the memory.

Cap output tokens per endpoint:

A model will write nine paragraphs where two were requested.

None of this is clever. It is dull platform work, which is why it waits for a bill.

Self-host or not

Regular readers know I am not a purist here.

Nearly everyone seriously ends up hybrid, and the platform layer is what makes that workable. Self-host the high-volume, predictable, privacy-sensitive work: embeddings, classification, extraction, internal knowledge assistants. That belongs to your own GPUs, where fixed cost beats per token pricing once volume settles. Keep frontier commercial models for low-volume work where the extra reasoning quality changes the outcome. The gateway lets you make that decision per workload rather than per company and revisit it each quarter as open models close the gap.

Starting from nothing, the order that works is: gateway and tracing first, even if you change nothing in the applications, because a week of visibility teaches more than six months of meetings. Cost attribution and caching second. Guardrails and redaction third. Evals fourth, and not much later than that.

Things I would not skip:

- Gateway from day one. Retrofitting is painful and turns political.
- Prompts in Git as versioned artefacts, not strings in code.
- Someone reads twenty real production outputs weekly. A human finds what metrics miss.
- Hard caps on dev spend, not alerts.
- Treat every retrieved document as hostile until proven otherwise. Keep a rollback path for prompt and model changes. They are deployments.

Where this leaves us

Pune is fine now. They capped the agent loop at four calls, moved

summarisation to a self-hosted Qwen model and turned on caching. They run at about 150,000 a month. The answers improved too, because the traces exposed a retrieval bug that had been there since the first release.

None of those fixes were difficult. All of them were visible the moment the team could see what their software was doing.

Getting a model to answer is solved by tools you can install in an afternoon. Running twenty of them without losing track of what they cost, what they tell customers and what they can touch is not solved. That is where the work has moved.

Nearly every piece you need is open source, self-hostable and already running in production somewhere. You need no budget and nobody's permission. A gateway, somewhere to put traces, and a willingness to look at what your systems are doing will take you a long way.

Put the layer in before you need it.



By: Vishal Tanna

The author is the CEO of TechCrave, a technology services company that helps enterprises adopt open source, cloud native, and AI-driven solutions. He has over 15 years of experience in software development and technology leadership, and is particularly interested in scalable architectures and digital transformation.

OSFY Magazine Attractions During 2026-27

Month	Theme
March 2026	Gen AI, Deep Learning and Machine Learning
April 2026	Open Source Cloud & Infrastructure
May 2026	Blockchain and Open Source
June 2026	Modern Software Development with Open Source Programming Languages
July 2026	Observability & Performance Security & Privacy in Open Source
August 2026	All About Data Management (Database management, Optimisation, Big Data, Data analysis, etc)
September 2026	DevOps, SRE & Platform Engineering
October 2026	Open Source and Quantum Computing
November 2026	Open Source for Edge, IoT & Embedded
December 2026	Cloud Special: Everything from Management to Implementation
January 2027	Python, Data Engineering and Open Data Stacks
February 2027	Open Source Communities & Careers The Future Of Open Source